

6차 회의록

WakeBuddy 품질관리 정리

1. 품질관리 적용 범위

WakeBuddy 프로젝트에서는 강의록의 품질관리 개념을 바탕으로, 현재 구현된 코드와 실제 테스트 결과를 중심으로 품질관리 활동을 정리하였다.

품질관리 대상은 다음과 같다.

구분	대상
Firebase 연결	<code>firebaseConfig.ts</code>
사용자 인증	<code>AuthService.ts</code> , <code>LoginScreen.tsx</code> , <code>SignUpScreen.tsx</code>
데이터 모델	<code>User</code> , <code>Alarm</code> , <code>FriendRequest</code> , <code>Friendship</code>
친구 관계 관리	<code>FriendService.ts</code> , <code>FriendRequestService.ts</code>
알람 권한 검증	<code>PermissionService.ts</code>
알람 시간 계산	<code>AlarmDateUtils.ts</code>
알림 예약 및 동기화	<code>AlarmService.ts</code> , <code>NotificationService.ts</code>
화면 전환	<code>App.tsx</code> , <code>AppHomeScreen.tsx</code>
테스트 코드	<code>test</code> 폴더의 Jest 테스트 파일

2. 품질관리 기준

강의록에서 품질관리는 소프트웨어 제품이나 개발 과정이 정해진 요구사항과 기준에 맞는지 확인하고 개선하는 활동으로 설명된다.

WakeBuddy에서는 다음 기능을 중심으로 품질관리 기준을 잡았다.

품질관리 기준	프로젝트 적용
요구사항 적합성	로그인, 회원가입, 알람 생성, 친구 알람 생성 기능이 요구사항대로 동작하는지 확인
정확성	알람 시간이 반복 유형에 따라 올바르게 계산되는지 확인
신뢰성	로그인 상태 감지 후 활성 알람을 동기화하는지 확인
보안성	친구가 아닌 사용자에게 알람을 생성하지 못하도록 권한 검증

품질관리 기준	프로젝트 적용
유지보수성	화면, 서비스, 모델, 유틸 코드를 역할별로 분리
사용성	로그인, 회원가입, 홈, 알람, 친구, 설정 화면이 구분되어 있는지 확인

3. 품질보증 활동

강의록에서 품질보증은 개발 과정에서 정해진 표준과 절차가 잘 지켜지는지 확인하는 활동이다.

WakeBuddy 코드에서 확인한 품질보증 활동은 다음과 같다.

항목	적용 내용
코드 역할 분리	<code>screens</code> , <code>services</code> , <code>models</code> , <code>utils</code> , <code>firebase</code> 로 역할 분리
데이터 구조 정의	<code>User</code> , <code>Alarm</code> , <code>FriendRequest</code> , <code>Friendship</code> 타입 정의
인증 로직 분리	로그인, 회원가입, 로그아웃, 로그인 상태 감지를 <code>AuthService</code> 에서 처리
친구 로직 분리	친구 조회, 친구 여부 확인, 친구 삭제, 친구 관계 생성을 <code>FriendService</code> 에서 처리
권한 로직 분리	알람 생성, 조회, 수정, 삭제 권한을 <code>PermissionService</code> 에서 확인
알람 시간 계산 분리	반복 없음, 매일 반복, 요일 반복 계산을 <code>AlarmDateUtils</code> 에서 처리
Firebase 설정 분리	Firebase 설정값을 환경변수에서 가져오도록 구성
주석 작성	주요 함수와 파일에 동작 목적을 설명하는 주석 작성

4. 코드 구조 점검

현재 프로젝트는 기능별로 파일이 분리되어 있다.

```
src/
├─ firebase/
│   └─ firebaseConfig.ts
├─ models/
│   ├── User.ts
│   ├── Alarm.ts
│   ├── FriendRequest.ts
│   └─ Friendship.ts
├─ screens/
│   ├── MainScreen.tsx
│   ├── LoginScreen.tsx
│   └─ SignUpScreen.tsx
```

```

|   └─ AppHomeScreen.tsx
|   └─ SettingsScreen.tsx
|
|   └─ services/
|       └─ AuthService.ts
|       └─ AlarmService.ts
|       └─ FriendService.ts
|       └─ FriendRequestService.ts
|       └─ NotificationService.ts
|       └─ PermissionService.ts
|
|   └─ utils/
|       └─ AlarmDateUtils.ts
|       └─ AuthValidationUtils.ts
|       └─ alarmSchedule.ts

```

코드 구조 점검 결과는 다음과 같다.

점검 항목	확인 내용
화면 코드 분리	화면 UI는 <code>screens</code> 폴더에서 관리
기능 로직 분리	인증, 친구, 알람, 알림, 권한 기능은 <code>services</code> 에서 관리
데이터 타입 분리	사용자, 알람, 친구 요청, 친구 관계 타입은 <code>models</code> 에서 관리
공동 함수 분리	알람 시간 계산과 입력값 검증은 <code>utils</code> 에서 관리
Firebase 설정 분리	Firebase 초기화는 <code>firebaseConfig.ts</code> 에서 관리

5. Firebase 설정 점검

Firebase 연결 설정은 `firebaseConfig.ts` 에서 관리한다.

```

const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};

const app = initializeApp(firebaseConfig);

export const auth = getAuth(app);
export const db = getFirestore(app);

```

점검 항목	확인 내용
Firebase 앱 초기화	<code>initializeApp(firebaseConfig)</code> 사용
인증 기능 연결	<code>getAuth(app)</code> 사용
Firestore 연결	<code>getFirestore(app)</code> 사용
설정값 관리	<code>.env</code> 환경변수 기반으로 Firebase 설정값 관리

6. 데이터 모델 점검

6.1 User 모델

```
exporttypeUser= {
  uid:string;
  email:string;
  displayName:string;
  createdAt>Date;
};
```

6.2 Alarm 모델

```
exporttypeAlarm= {
  alarmId:string;
  ownerId:string;
  creatorId:string;
  title:string;
  alarmDate?:Date;
  alarmTime>Date;
  repeatType:RepeatType;
  repeatDays:Weekday[];
  isActive:boolean;
  notificationId?:string;
  createdAt>Date;
  updatedAt>Date;
};
```

6.3 FriendRequest 모델

```
exporttypeFriendRequestStatus=
| "pending"
| "accepted"
| "rejected"
| "canceled";
```

```
exporttypeFriendRequest= {
  requestId:string;
  senderId:string;
  receiverId:string;
  status:FriendRequestStatus;
  createdAt:Date;
  updatedAt:Date;
};
```

6.4 Friendship 모델

```
exporttypeFriendship= {
  friendshipId:string;
  userIds:string[];
  createdBy:string;
  createdAt:Date;
};
```

6.5 점검 결과

점검 항목	확인 내용
사용자 데이터	uid, email, displayName, createdAt으로 구성
알람 데이터	ownerId와 creatorId로 알람 대상자와 생성자 구분
반복 알람	repeatType과 repeatDays로 반복 유형 관리
알림 예약	notificationId 필드로 예약된 알림 ID 관리 가능
친구 요청	pending, accepted, rejected, canceled 상태 구분
친구 관계	userIds 배열로 두 사용자 관계 저장

7. 인증 기능 점검

인증 기능은 `AuthService.ts` 에서 관리한다.

7.1 입력값 검증

입력값 검증 로직은 `AuthValidationUtils.ts` 로 분리하였다.

```
exportfunctionvalidateLoginInput(email:string,password:string):void {
  if (!email.trim()) {
    thrownewError("이메일을 입력해주세요.");
  }

  if (!password.trim()) {
```

```
thrownewError("비밀번호를 입력해주세요.");
}
}
```

```
exportfunctionvalidateSignUpInput(
  email:string,
  password:string,
  displayName:string,
):void {
  if (!email.trim()) {
    thrownewError("이메일을 입력해주세요.");
  }

  if (!password.trim()) {
    thrownewError("비밀번호를 입력해주세요.");
  }

  if (!displayName.trim()) {
    thrownewError("이름을 입력해주세요.");
  }
}
```

7.2 회원가입 처리

```
validateSignUpInput(email,password,displayName);

constuserCredential=awaitcreateUserWithEmailAndPassword(
  auth,
  email,
  password,
);

constfirebaseUser=userCredential.user;

awaitsetDoc(doc(db,"users",firebaseUser.uid), {
  uid:firebaseUser.uid,
  email:firebaseUser.email??email,
  displayName,
  createdAt:serverTimestamp(),
});
```

7.3 로그인 처리

```
validateLoginInput(email,password);

constuserCredential=awaitsignInWithEmailAndPassword(
  auth,
  email,
```

```
password,
);

const firebaseUser = userCredential.user;
const userDoc = await getDoc(doc(db, "users", firebaseUser.uid));
```

7.4 점검 결과

점검 항목	확인 내용
회원가입 입력 검증	이메일, 비밀번호, 이름 미입력 시 오류 처리
로그인 입력 검증	이메일, 비밀번호 미입력 시 오류 처리
Firebase Auth 사용	회원가입과 로그인에 Firebase Authentication 사용
사용자 정보 저장	회원가입 후 Firestore <code>users</code> 컬렉션에 사용자 정보 저장
사용자 정보 조회	로그인 후 Firestore에서 사용자 프로필 조회
로그인 상태 감지	<code>onAuthStateChanged</code> 로 로그인 상태 변화 확인

8. 친구 기능 점검

친구 관계 관리는 `FriendService.ts` 에서 처리한다.

8.1 친구 목록 조회

```
const friendshipQuery = query(
  collection(db, FRIENDSHIPS_COLLECTION),
  where("userIds", "array-contains", userId),
);

const snapshot = await getDocs(friendshipQuery);
```

8.2 친구 여부 확인

```
async isFriend(userId: string, friendId: string): Promise<boolean> {
  if (!userId || !friendId) {
    return false;
  }

  const friendshipQuery = query(
    collection(db, FRIENDSHIPS_COLLECTION),
    where("userIds", "array-contains", userId),
  );

  const snapshot = await getDocs(friendshipQuery);
```

```

returnsnapshot.docs.some((friendshipDoc) => {
  constdata=friendshipDoc.data();
  constuserIds=data.userIds?? [];

  returnuserIds.includes(friendId);
});
}

```

8.3 친구 관계 생성

```

constalreadyFriend=awaitthis.isFriend(userId,friendId);

if (alreadyFriend) {
  return;
}

constfriendshipRef=doc(collection(db,FRIENDSHIPS_COLLECTION));

awaitsetDoc(friendshipRef, {
  userIds: [userId,friendId],
  createdBy,
  createdAt:serverTimestamp(),
});

```

8.4 점검 결과

점검 항목	확인 내용
친구 목록 조회	현재 사용자 uid가 포함된 friendship 문서 조회
친구 정보 조회	상대 uid를 기준으로 users 문서 조회
친구 여부 확인	friendship 문서의 userIds 배열에 상대 uid가 포함되는지 확인
친구 관계 생성	이미 친구인 경우 중복 생성하지 않음
친구 삭제	두 사용자가 포함된 friendship 문서를 찾아 삭제

9. 권한 검증 점검

권한 검증은 `PermissionService.ts` 에서 처리한다.

9.1 알람 생성 권한

```

asynccanCreateAlarmForUser(
  currentUserId:string,
  targetUserId:string,

```



```

):Promise<boolean> {
  if (!currentUserId||!targetUserId) {
    returnfalse;
  }

  if (currentUserId===targetUserId) {
    returntrue;
  }

  returnFriendService.isFriend(currentUserId,targetUserId);
}

```

9.2 알람 조회 권한

```

canViewAlarm(currentUserId: string,alarm:Alarm):boolean{
  return (
    currentUserId===alarm.ownerId||currentUserId===alarm.creatorId
  );
}

```

9.3 알람 수정 권한

```

canEditAlarm(currentUserId: string,alarm:Alarm):boolean{
  return (
    currentUserId===alarm.ownerId||currentUserId===alarm.creatorId
  );
}

```

9.4 알람 삭제 권한

```

canDeleteAlarm(currentUserId: string,alarm:Alarm):boolean{
  return (
    currentUserId===alarm.ownerId||currentUserId===alarm.creatorId
  );
}

```

9.5 점검 결과

점검 항목	확인 내용
본인 알람 생성	currentUserId와 targetUserId가 같으면 허용
친구 알람 생성	다른 사용자에게 생성할 경우 친구 관계 확인
비정상 사용자 값	currentUserId 또는 targetUserId가 없으면 false 반환

점검 항목	확인 내용
알람 조회	알람 대상자 또는 생성자만 조회 가능
알람 수정	알람 대상자 또는 생성자만 수정 가능
알람 삭제	알람 대상자 또는 생성자만 삭제 가능

10. 알람 시간 계산 점검

알람 시간 계산은 `AlarmDateUtils.ts` 에서 처리한다.

10.1 요일 변환

```
exportfunctionweekdayToNumber(weekday:Weekday):number {
  constweekdayMap:Record<Weekday,number>= {
    sun:0,
    mon:1,
    tue:2,
    wed:3,
    thu:4,
    fri:5,
    sat:6,
  };

  returnweekdayMap[weekday];
}
```

10.2 반복 없음 알람

```
exportfunctiongetOneTimeAlarmDate(alarm:Alarm):Date {
  constnow=newDate();
  constnext=newDate();

  next.setHours(alarm.alarmTime.getHours());
  next.setMinutes(alarm.alarmTime.getMinutes());
  next.setSeconds(0);
  next.setMilliseconds(0);

  if (next.getTime()<=now.getTime()) {
    thrownewError("현재 시간 이후의 알람 시간을 선택해주세요.");
  }

  returnnext;
}
```

10.3 매일 반복 알람

```

export function getNextDailyAlarmDate(alarm: Alarm): Date {
    const now = new Date();
    const next = new Date();

    next.setHours(alarm.alarmTime.getHours());
    next.setMinutes(alarm.alarmTime.getMinutes());
    next.setSeconds(0);
    next.setMilliseconds(0);

    if (next.getTime() <= now.getTime()) {
        next.setDate(next.getDate() + 1);
    }

    return next;
}

```

10.4 요일 반복 알람

```

export function getNextWeeklyAlarmDate(alarm: Alarm, weekday: Weekday): Date {
    const now = new Date();
    const targetDay = weekdayToNumber(weekday);

    const next = new Date();
    next.setHours(alarm.alarmTime.getHours());
    next.setMinutes(alarm.alarmTime.getMinutes());
    next.setSeconds(0);
    next.setMilliseconds(0);

    const currentDay = next.getDay();
    let daysUntilTarget = targetDay - currentDay;

    if (daysUntilTarget < 0) {
        daysUntilTarget += 7;
    }

    next.setDate(next.getDate() + daysUntilTarget);

    if (next.getTime() <= now.getTime()) {
        next.setDate(next.getDate() + 7);
    }

    return next;
}

```

10.5 점검 결과

점검 항목	확인 내용
요일 변환	sun~sat 값을 JavaScript Date 요일 숫자로 변환

점검 항목	확인 내용
반복 없음	현재 시간 이후의 알람만 설정 가능
매일 반복	오늘 시간이 지났으면 다음 날로 계산
요일 반복	선택 요일 기준 다음 실행 날짜 계산
시간 단위 처리	초와 밀리초를 0으로 초기화

11. 화면 전환 점검

화면 전환은 `App.tsx` 와 `AppHomeScreen.tsx` 에서 관리한다.

11.1 앱 전체 화면 상태

```
export type ScreenName = "main" | "login" | "signup" | "home" | "alarmForm";

const [screen, setScreen] = useState<ScreenName>("main");
```

11.2 로그인 상태 감지

```
useEffect(() => {
  const unsubscribe = AuthService.watchAuthState(async (user) => {
    setCurrentUser(user);
    setIsAuthLoading(false);

    if (user) {
      setScreen("home");
    }

    try {
      await AlarmService.syncMyActiveAlarms(user.uid);
    } catch (error) {
      console.log("알람 동기화 실패:", error);
    }
    } else {
      setScreen("main");
    }
  });

  return unsubscribe;
}, []);
```

11.3 홈 탭 전환

```
type HomeTab = "alarm" | "friend" | "settings";
```

```
const [tab,setTab]=useState<HomeTab>("alarm");
```

```
{tab==="alarm"&& (
  <AlarmListScreen
    currentUser={currentUser}
    goCreateAlarm={goCreateAlarm}
    goEditAlarm={goEditAlarm}
  />
)}

{tab==="friend"&& (
  <FriendTabScreen
    currentUser={currentUser}
    goCreateFriendAlarm={goCreateFriendAlarm}
  />
)}

{tab==="settings"&&<SettingsScreencurrentUser={currentUser}/>}
```

11.4 점검 결과

점검 항목	확인 내용
로그인 전 화면	Main, Login, SignUp 화면 사용
로그인 후 화면	Home 화면으로 이동
알람 생성 화면	alarmForm 상태로 이동
홈 탭	알람, 친구, 설정 탭으로 구분
로그인 상태 반영	Firebase 로그인 상태에 따라 화면 전환
알람 동기화	로그인 상태 확인 후 활성 알람 동기화 호출

12. 알림 설정 점검

알림 설정은 `app.json` 과 `NotificationService` 연동 부분에서 확인하였다.

12.1 app.json 알림 설정

```
"plugins": [
  "@react-native-community/datetimepicker",
  [
    "expo-notifications",
    {
      "sounds": [
        "./assets/sounds/alarm_29s.wav"
      ],
    },
  ],
]
```

```

    "enableBackgroundRemoteNotifications": false
  }
]
]

```

12.2 알림 응답 리스너

```

useEffect(() => {
  const unsubscribe =
    NotificationService.registerNotificationResponseListener();

  return unsubscribe;
}, []);

```

12.3 점검 결과

점검 항목	확인 내용
알림 플러그인	<code>expo-notifications</code> 사용
알림음 파일	<code>alarm_29s.wav</code> 등록
시간 선택	<code>@react-native-community/datetimepicker</code> 사용
알림 응답 처리	알림 클릭 시 처리할 리스너 등록

13. 인스펙션 체크리스트

강의록의 인스펙션 개념을 기준으로 코드 검토 항목을 정리하였다.

구분	점검 항목	결과
Firebase	Firebase 앱 초기화 여부	확인
Firebase	Auth, Firestore 연결 여부	확인
Firebase	설정값 환경변수 분리 여부	확인
인증	회원가입 입력값 검증 여부	확인
인증	로그인 입력값 검증 여부	확인
인증	회원가입 후 사용자 정보 저장 여부	확인
인증	로그인 상태 감지 여부	확인
모델	User 타입 정의 여부	확인
모델	Alarm 타입 정의 여부	확인
모델	FriendRequest 타입 정의 여부	확인

구분	점검 항목	결과
모델	Friendship 타입 정의 여부	확인
친구	친구 목록 조회 로직 존재 여부	확인
친구	친구 여부 확인 로직 존재 여부	확인
친구	친구 관계 중복 생성 방지 여부	확인
권한	본인 알람 생성 허용 여부	확인
권한	친구 알람 생성 전 친구 여부 확인 여부	확인
권한	알람 조회/수정/삭제 권한 확인 여부	확인
알람 시간	반복 없음 알람에서 과거 시간 방지 여부	확인
알람 시간	매일 반복 알람 날짜 계산 여부	확인
알람 시간	요일 반복 알람 날짜 계산 여부	확인
화면	로그인 전/후 화면 분기 여부	확인
화면	홈 탭 전환 여부	확인
알림	expo-notifications 설정 여부	확인
알림	알림음 파일 등록 여부	확인

14. 품질 측정 메트릭

강의록의 품질 측정과 메트릭 개념에 맞춰, 실제 코드와 테스트로 확인 가능한 항목만 정리하였다.

메트릭	측정 방법	관련 코드
로그인 입력값 검증 여부	빈 이메일/비밀번호 입력 시 오류 발생 확인	<code>AuthValidationUtils.ts</code>
회원가입 입력값 검증 여부	빈 이메일/비밀번호/이름 입력 시 오류 발생 확인	<code>AuthValidationUtils.ts</code>
사용자 정보 저장 여부	회원가입 후 Firestore users 문서 생성 확인	<code>AuthService.ts</code>
친구 여부 확인 여부	두 사용자의 친구 관계 여부 확인	<code>FriendService.ts</code>
알람 생성 권한 검증 여부	본인/친구/비친구 대상 권한 결과 확인	<code>PermissionService.ts</code>
알람 조회 권한 검증 여부	ownerId/creatorId 기준 조회 가능 여부 확인	<code>PermissionService.ts</code>
알람 수정 권한 검증 여부	ownerId/creatorId 기준 수정 가능 여부 확인	<code>PermissionService.ts</code>

메트릭	측정 방법	관련 코드
알람 삭제 권한 검증 여부	ownerId/creatorId 기준 삭제 가능 여부 확인	<code>PermissionService.ts</code>
반복 없음 알람 계산 여부	현재 시간 이전이면 오류 발생 확인	<code>AlarmDateUtils.ts</code>
매일 반복 알람 계산 여부	지난 시간이면 다음 날로 계산되는지 확인	<code>AlarmDateUtils.ts</code>
요일 반복 알람 계산 여부	선택 요일 기준 다음 실행 날짜 계산 확인	<code>AlarmDateUtils.ts</code>
로그인 상태 화면 전환 여부	로그인/로그아웃 시 화면 변경 확인	<code>App.tsx</code>
알람 동기화 호출 여부	로그인 후 활성 알람 동기화 호출 확인	<code>App.tsx</code>
알림음 설정 여부	app.json에 알림음 파일 등록 확인	<code>app.json</code>

15. 테스트 코드 추가

품질검증을 위해 Jest 기반 단위 테스트를 추가하였다.

15.1 추가한 테스트 파일

```
test/
├─ AuthValidationUtils.test.ts
├─ AlarmDateUtils.test.ts
└─ PermissionService.test.ts
```

15.2 테스트 대상

테스트 파일	테스트 대상
<code>AuthValidationUtils.test.ts</code>	로그인/회원가입 입력값 검증
<code>AlarmDateUtils.test.ts</code>	반복 없음, 매일 반복, 요일 반복 알람 시간 계산
<code>PermissionService.test.ts</code>	본인/친구/비친구 알람 권한, 조회/수정/삭제 권한

16. 테스트 결과

Jest 테스트 실행 결과는 다음과 같다.

```
PASS test/AuthValidationUtils.test.ts
PASS test/AlarmDateUtils.test.ts
PASS test/PermissionService.test.ts

Test Suites: 3 passed, 3 total
```



```
Tests:      35 passed, 35 total
Snapshots:  0 total
Time:       2.824 s
Ran all test suites.
```

16.1 테스트 결과 정리

항목	결과
테스트 파일 수	3개
통과한 테스트 파일	3개
전체 테스트 케이스	35개
통과한 테스트 케이스	35개
실패한 테스트 케이스	0개

17. 품질검증 항목 정리

구분	검증 내용	결과
입력값 검증	로그인/회원가입 필수 입력값 검증	통과
알람 시간 계산	반복 없음 알람에서 과거 시간 방지	통과
알람 시간 계산	매일 반복 알람의 다음 실행 날짜 계산	통과
알람 시간 계산	요일 반복 알람의 다음 실행 날짜 계산	통과
권한 검증	본인에게 알람 생성 가능	통과
권한 검증	친구에게 알람 생성 가능	통과
권한 검증	비친구에게 알람 생성 차단	통과
권한 검증	알람 대상자/생성자만 조회 가능	통과
권한 검증	알람 대상자/생성자만 수정 가능	통과
권한 검증	알람 대상자/생성자만 삭제 가능	통과

18. 프로세스 개선

강의록의 프로세스 개선 개념에 맞춰, 테스트 과정에서 추가하거나 정리한 내용은 다음과 같다.

개선 항목	내용
입력값 검증 분리	<code>AuthService.ts</code> 안에 있던 입력값 검증을 <code>AuthValidationUtils.ts</code> 로 분리

개선 항목	내용
테스트 폴더 추가	루트에 <code>test</code> 폴더를 생성하여 테스트 코드 관리
알람 시간 계산 테스트 추가	반복 없음, 매일 반복, 요일 반복 알람 계산 결과 검증
권한 검증 테스트 추가	본인/친구/비친구 권한 처리 결과 검증
Jest 설정 추가	<code>jest.config.js</code> 생성
테스트 스크립트 추가	<code>package.json</code> 에 <code>"test": "jest"</code> 추가
테스트 라이브러리 버전 정리	Jest 29 버전으로 맞춰 테스트 실행 오류 해결